

Perbandingan Subbab 3.2.4.1: Versi Saat Ini dan Versi Usulan

Fokus perbandingan: struktur penjelasan, alur narasi, dan pemetaan seluruh inti Subbab 4.1.3 ke rancangan Subbab 3.2.4.1.

A. Versi 3.2.4.1 Saat Ini

Perancangan Komponen *Retrieval* pada Sistem

Perancangan komponen *retrieval* pada sistem dilakukan dengan mengimplementasikan pendekatan *hybrid* yang menggabungkan metode berbasis representasi semantik (*dense*) dan pencocokan kata kunci (*sparse*). Komponen pertama adalah transformasi dokumen yang mencakup pengolahan data kontrol standar ISO dari sumber basis pengetahuan menjadi representasi teks yang terstruktur dan siap digunakan dalam proses *retrieval*. Komponen kedua adalah transformasi *query* yaitu proses merepresentasikan *query* ke dalam bentuk yang dapat diproses oleh sistem, baik melalui pendekatan semantik (*dense*) maupun berbasis kata kunci (*sparse*). Komponen ketiga adalah *scoring*, yaitu mekanisme pengolahan dan penggabungan skor dari kedua pendekatan untuk menghasilkan nilai akhir yang digunakan dalam menentukan relevansi dan peringkat dokumen hasil pencarian.

Berdasarkan alur yang ditunjukkan pada gambar komponen *hybrid retrieval*, proses *retrieval* dapat diuraikan ke dalam tiga komponen utama sebagai berikut.

1. Transformasi Dokumen

Pada tahap ini, dokumen direpresentasikan dalam dua pendekatan yang berbeda untuk mendukung proses *retrieval*. Pada jalur *dense*, setiap dokumen diubah menjadi representasi numerik berupa vektor melalui proses *embedding*, sehingga memungkinkan perhitungan kesamaan berbasis makna. Sementara itu, pada jalur *sparse*, dokumen diubah menjadi representasi leksikal melalui proses tokenisasi. Kedua representasi ini selanjutnya digunakan dalam proses perhitungan relevansi.

2. Transformasi *Query*

Query yang diberikan pengguna diproses dengan pendekatan serupa, yaitu diubah menjadi representasi vektor melalui *embedding* pada jalur *dense* dan menjadi representasi token pada jalur *sparse*. Representasi *query* ini kemudian dibandingkan dengan dokumen yang telah diproses sebelumnya. Pada jalur *dense*, perbandingan dilakukan menggunakan *dot product*, sedangkan pada jalur *sparse* menggunakan BM25. Hasil dari proses ini adalah skor relevansi terhadap seluruh dokumen, yang dalam penelitian ini berjumlah 93 skor untuk masing-masing kontrol.

3. *Scoring* dan Pemilihan Kandidat

Skor yang dihasilkan dari kedua pendekatan selanjutnya dinormalisasi untuk memastikan kesetaraan skala dari perhitungan tiap metode. Setelah itu, skor digabungkan menggunakan mekanisme *hybrid scoring* untuk menghasilkan nilai akhir yang merepresentasikan tingkat relevansi dokumen. Berdasarkan skor tersebut, dokumen diurutkan dan dipilih sejumlah kandidat teratas, yaitu top@3. Metadata dari dokumen terpilih kemudian diekstraksi dan digunakan sebagai *input* pada tahap selanjutnya, yaitu proses generasi menggunakan LLM pada *pipeline Retrieval-Augmented Generation*.

B. Versi Usulan dengan Narasi dan Subpoin Teknis (Grup: Knowledge Base, Query Processing, Hybrid Scoring)

Perancangan Komponen *Retrieval* pada Sistem

Perancangan komponen *retrieval* pada sistem dilakukan menggunakan pendekatan *hybrid retrieval* yang menggabungkan jalur *dense retrieval* dan *sparse retrieval*. Jalur *dense* digunakan untuk menangkap kemiripan semantik antara *query* dan dokumen melalui representasi *embedding*, sedangkan jalur *sparse* digunakan untuk menangkap kecocokan leksikal berbasis kata kunci menggunakan BM25. Kedua jalur tersebut kemudian digabungkan melalui proses normalisasi dan pembobotan skor untuk menghasilkan kandidat dokumen yang paling relevan.

Berdasarkan rancangan tersebut, komponen *retrieval* dibagi ke dalam tiga bagian utama, yaitu *knowledge base* sebagai tahap inialisasi indeks dokumen, *query processing* sebagai tahap pemrosesan *query* dan perhitungan skor, serta *hybrid scoring* sebagai tahap penggabungan skor dan pemilihan kandidat.

1. *Knowledge Base* — Fase Inialisasi

Tahap *knowledge base* merupakan fase inialisasi yang dijalankan sekali saat sistem dimulai. Pada tahap ini, data kontrol ISO berbentuk JSON dimuat dan dikonversi menjadi representasi teks terstruktur. Representasi tersebut kemudian diproses pada dua jalur secara paralel: jalur *dense* membentuk *embedding* dokumen, sedangkan jalur *sparse* membangun indeks BM25. Kedua indeks yang dihasilkan disimpan dalam memori dan digunakan ulang pada setiap pemrosesan *query*.

- a. **Representasi Teks:** Menggunakan *function* `control_to_text(control: dict) -> str` untuk menggabungkan field JSON menjadi satu teks utuh. Representasi ini menjadi masukan utama bagi proses *embedding* dan tokenisasi agar konteks dokumen dapat diproses secara konsisten.

- b. **Inisiasi Model dan Dimensi *Embedding*:** Menggunakan *function* `get_model()` -> `SentenceTransformer` untuk memuat model *paraphrase-multilingual-MiniLM-L12-v2* dengan mekanisme *lazy loading*. Selanjutnya, `get_embedding_dim()` -> `int` digunakan untuk memperoleh dimensi *embedding* sebesar 384 dimensi.
- c. **Pembentukan *Embedding* Dokumen:** Menggunakan *function* `encode_texts(texts: list[str])` -> `np.ndarray` untuk mengubah seluruh teks kontrol ISO menjadi vektor *embedding*. Hasilnya berupa *array* NumPy berdimensi (93, 384) dengan tipe data `float32`.
- d. **Normalisasi *Embedding* Dokumen:** Menggunakan *function* `normalize(vectors: np.ndarray)` -> `np.ndarray` untuk melakukan normalisasi L2 pada vektor *embedding*. Normalisasi ini membuat perhitungan *dot product* setara dengan *cosine similarity* dan menjaga stabilitas ketika terdapat vektor bernilai nol.
- e. **Penyimpanan *Embedding* Dokumen:** *Embedding* dokumen disimpan pada atribut `HybridRetriever.doc_embeddings` secara *in-memory*. Penyimpanan dilakukan dalam bentuk *array* NumPy dan dibangun ulang setiap kali sistem dijalankan.
- f. **Preprocessing Teks untuk BM25:** Menggunakan *function* `tokenize(text: str)` -> `list[str]` untuk mengubah teks dokumen menjadi daftar *token*. Proses ini dilakukan dengan *lowercase* dan pengambilan karakter alfanumerik menggunakan *regex* `[A-Za-z0-9]+`.
- g. **Pembangunan Indeks BM25:** Pada saat `BM25Retriever` diinisialisasi, seluruh representasi teks kontrol ISO diproses menggunakan `tokenize()`, kemudian sistem menghitung panjang dokumen (`doc_lens`), *document frequency* (`_compute_doc`) dan *inverse document frequency* menggunakan formula Okapi BM25, yaitu $\log(1 + (N - df + 0.5) / (df + 0.5))$. Indeks yang dihasilkan disimpan dalam memori dan siap digunakan pada tahap *query processing*.

2. Query Processing — Pemrosesan Query dan Perhitungan Skor

Tahap *query processing* dijalankan setiap kali pengguna memberikan *query*. Pada tahap ini, *query* diproses secara paralel pada dua jalur: jalur *dense* mengubah *query* menjadi vektor *embedding* dan menghitung kemiripan semantik terhadap seluruh dokumen, sedangkan jalur *sparse* menokenisasi *query* dan menghitung skor BM25. Hasil dari kedua jalur adalah masing-masing 93 skor relevansi yang akan digunakan pada tahap *hybrid scoring*.

- a. **Pembentukan *Embedding Query*:** Menggunakan *function* `encode_texts([query])` untuk mengubah *query* menjadi vektor *embedding* berdimensi 384. Karena fungsi yang sama digunakan untuk dokumen dan *query*, keduanya direpresentasikan dalam ruang vektor yang sama.

- b. **Perhitungan Dense Score:** Menggunakan *function* `_dense_scores(query: str) -> np.ndarray` untuk menghitung *dot product* antara vektor *query* dan *embedding* seluruh dokumen yang telah disimpan pada fase inisialisasi. Karena kedua vektor telah dinormalisasi, nilai *dot product* ini setara dengan *cosine similarity*.
- c. **Tokenisasi Query:** Pada jalur *sparse*, *query* diproses menggunakan `tokenize()` yang sama dengan dokumen. Konsistensi tokenisasi ini memastikan bahwa proses pencocokan *term* pada BM25 berjalan dengan aturan yang sama.
- d. **Perhitungan Sparse Score:** Menggunakan *function* `score_query(query: str) -> list[float]` untuk menghitung skor BM25 *query* terhadap seluruh dokumen berdasarkan kombinasi *term frequency*, IDF, dan normalisasi panjang dokumen.

3. Hybrid Scoring — Penggabungan Skor dan Pemilihan Kandidat

Tahap *hybrid scoring* menggabungkan skor dari jalur *dense* dan *sparse* menjadi satu nilai akhir yang merepresentasikan relevansi dokumen. Skor dari kedua jalur terlebih dahulu dinormalisasi agar memiliki skala yang sama, kemudian digabungkan menggunakan *weighted sum*. Berdasarkan skor hasil penggabungan, sistem mengurutkan dokumen dan memilih kandidat teratas yang digunakan sebagai konteks untuk tahap generasi.

- a. **Normalisasi Skor:** Menggunakan *function* `_minmax(scores: list[float]) -> np.ndarray` untuk menormalisasi skor *dense* dan *sparse* ke rentang 0 sampai 1. Normalisasi dilakukan secara terpisah karena kedua skor memiliki skala yang berbeda.
- b. **Fusion Score:** Menggunakan metode *weighted sum* untuk menggabungkan skor *dense* dan *sparse* yang telah dinormalisasi. Formula yang digunakan adalah $hybrid_score = w_{dense} \times dense_norm + w_{sparse} \times sparse_norm$, dengan bobot bawaan $w_{dense} = 0,6$ dan $w_{sparse} = 0,4$.
- c. **Final Ranking:** Setelah *hybrid score* diperoleh, sistem mengurutkan seluruh dokumen secara *descending* menggunakan `np.argsort` dan memilih 3 dokumen dengan skor tertinggi.
- d. **Pembentukan Hasil:** Menggunakan *function* `_build_result(idx)` untuk mengambil metadata kontrol terpilih, yaitu `control_id`, `title`, `objective`, dan `description`, serta skor *dense*, *sparse*, dan *hybrid*. Hasil ini dikirimkan sebagai konteks ke LLM pada tahap generasi.

C. Checklist Pemetaan Inti 4.1.3

Sumber	Poin inti	Masuk ke versi usulan
4.1.3.1 Dense	Representasi teks; inisiasi model; dimensi <i>embedding</i> ; pembentukan <i>embedding</i> ; normalisasi; penyimpanan <i>embedding</i> dokumen.	Ya, bagian <i>Knowledge Base</i> (a–e) dan <i>Query Processing</i> (a–b).
4.1.3.2 Sparse	<i>Preprocessing</i> ; tokenisasi dokumen; panjang dokumen; <i>document frequency</i> ; IDF; skor BM25.	Ya, bagian <i>Knowledge Base</i> (f–g) dan <i>Query Processing</i> (c–d).
4.1.3.3 Hybrid	Inisialisasi <i>HybridRetriever</i> ; <i>dense score</i> ; <i>sparse score</i> ; <i>method_order</i> ; <i>candidate merging</i> ; normalisasi; <i>fusion score</i> ; <i>final ranking</i> .	Ya, bagian <i>Hybrid Scoring</i> (a–d).

D. Rekomendasi

Versi usulan lebih direkomendasikan untuk menggantikan isi 3.2.4.1 karena struktur ketiga grup (*Knowledge Base*, *Query Processing*, *Hybrid Scoring*) selaras dengan diagram hybrid retrieval yang memperlihatkan ketiga bagian tersebut secara paralel dan sekuensial. Seluruh inti implementasi dari 4.1.3.1, 4.1.3.2, dan 4.1.3.3 tetap terwakili. Format narasi singkat diikuti subpoin teknis juga membuat pembahasan lebih mudah dibaca dibanding memindahkan seluruh daftar 4.1.3 secara mentah.

E. Perbandingan Subbab 3.2.4.2 dengan Pengembangan dari 4.1.4

E.1 Versi 3.2.4.2 Saat Ini

Perancangan Komponen *Rerank* pada Sistem

Pengembangan sistem *Retrieval-Augmented Generation* dengan menambahkan tahapan *reranking* berbasis *cross-encoder* dilakukan untuk meningkatkan kualitas hasil *retrieval*. Secara umum, *pipeline* ini terdiri dari dua tahap utama, yaitu melanjutkan hasil dari komponen *hybrid retrieval* sebelumnya dan memproses kandidat tersebut lebih lanjut untuk memperoleh urutan dokumen berdasarkan relevansi yang lebih mendalam.

Berdasarkan rancangan saat ini, komponen *rerank* diuraikan menjadi dua tahap utama sebagai berikut.

1. Tahap *Hybrid Retrieval*

Tahap ini menggunakan hasil dari proses *hybrid retrieval* sebagai sumber kandidat awal. Skor *dense* dan *sparse* digabungkan menggunakan skema pembobotan tertentu untuk menghasilkan skor *hybrid*. Berdasarkan skor tersebut, sejumlah kandidat dokumen dengan peringkat tertinggi dipilih sebagai *candidate pool* untuk tahap *reranking*.

2. *Cross-Encoder Reranking*

Tahap *reranking* mengambil *candidate pool* dari hasil *hybrid retrieval*. Setiap pasangan *query* dan dokumen diproses secara bersamaan oleh model *cross-encoder* BAAI/bge-reranker-v2-m3 untuk menghasilkan skor relevansi baru. Skor tersebut kemudian dinormalisasi dan dikombinasikan dengan skor *hybrid* menggunakan skema pembobotan sehingga diperoleh skor akhir yang lebih representatif. Berdasarkan skor ini, dipilih dokumen terbaik, yaitu top@3, sebagai konteks untuk tahap generasi jawaban.

E.2 Versi Usulan Berdasarkan Pemecahan Inti Subbab 4.1.4

Perancangan Komponen *Rerank* pada Sistem

Perancangan komponen *rerank* dilakukan untuk meningkatkan kualitas peringkat dokumen yang telah diperoleh dari tahap *hybrid retrieval*. Agar tetap selaras dengan struktur 3.2.4.2 yang sudah ada, rancangan ini sebaiknya tetap dibagi menjadi dua bagian utama, yaitu tahap *hybrid retrieval* dan tahap *cross-encoder reranking*. Detail dari 4.1.4.1 hingga 4.1.4.4 tidak perlu dijadikan empat bagian utama, melainkan dimasukkan sebagai subpoin teknis di bawah dua bagian tersebut.

1. Tahap *Hybrid Retrieval*

Tahap *hybrid retrieval* berfungsi sebagai tahap awal untuk menghasilkan daftar kandidat dokumen yang akan dievaluasi ulang oleh *reranker*. Pada tahap ini, sistem menggunakan kembali komponen *hybrid retrieval* dari RAG V1, memuat korpus kontrol ISO yang telah diperkaya, mengatur jumlah kandidat awal, serta membentuk *candidate pool* berdasarkan skor *hybrid*. Dengan demikian, kandidat yang masuk ke tahap *reranking* telah mempertimbangkan sinyal relevansi semantik dan leksikal.

- a. **Inisialisasi *Pipeline*:** Menggunakan *class* `RAGPipelineV3` untuk mengatur alur kerja mulai dari *retrieval*, *reranking*, hingga pembentukan konteks untuk generasi. Komponen ini menggunakan kembali `HybridRetriever` dari RAG V1 agar tahap *retrieval* tetap konsisten dengan rancangan *dense*, *sparse*, dan *hybrid retrieval* sebelumnya.
- b. **Penggunaan Korpus yang Diperkaya:** Sistem menggunakan berkas `iso_controls_enrich` sebagai sumber data kontrol ISO. Korpus ini memuat atribut dasar dan atribut tambahan seperti `keywords_en`, `keywords_id`, `audit_indicators_en`, dan `audit_indicators_id` untuk memperkaya sinyal semantik pada tahap *retrieval* dan *reranking*.
- c. **Konfigurasi Jumlah Kandidat:** Sistem menggunakan `CANDIDATE_POOL_SIZE` bernilai 10 sebagai jumlah kandidat awal dari *hybrid retrieval* dan `DEFAULT_TOP_K` bernilai 3 sebagai jumlah dokumen akhir setelah *reranking*. Konfigurasi ini memberi ruang kandidat yang cukup sebelum dilakukan evaluasi ulang.
- d. **Pemanggilan *Hybrid Retrieval*:** Menggunakan `retrieve()` pada `HybridRetriever` dengan `method_order=["hybrid"]` untuk mengambil hasil *hybrid* sebagai kandidat awal. Perhitungan skor *dense* dan *sparse* tetap dilakukan secara internal.
- e. **Pengaturan Bobot *Dense* dan *Sparse*:** Parameter `alpha_dense` dan `alpha_sparse` digunakan untuk mengatur kontribusi skor *dense* dan *sparse* dalam pembentukan `hybrid_score`. Skor ini menjadi dasar pengurutan kandidat tahap pertama.
- f. **Penyimpanan Informasi Kandidat:** Setiap kandidat menyimpan informasi kontrol ISO seperti `control_id`, `title`, `objective`, dan `description`, serta skor *dense*, *sparse*, dan *hybrid*. Informasi ini dipertahankan karena digunakan kembali pada tahap penggabungan skor setelah proses *reranking*.

2. *Cross-Encoder Reranking*

Tahap *cross-encoder reranking* berfungsi untuk mengevaluasi ulang kandidat dokumen dari tahap *hybrid retrieval*. Pada tahap ini, setiap pasangan *query* dan dokumen diproses secara bersamaan oleh model *cross-encoder* untuk menghasilkan skor relevansi baru. Skor tersebut kemudian dinormalisasi, digabungkan dengan skor *hybrid*, dan digunakan untuk menentukan urutan akhir dokumen.

- a. **Inisialisasi Model *Cross-Encoder*:** Menggunakan *function* `get_model()` -> `CrossEncoder` untuk memuat model *BAAI/bge-reranker-v2-m3* dengan mekanisme *lazy loading*. Model ini digunakan karena mendukung banyak bahasa, termasuk Bahasa Indonesia.
- b. **Alternatif Model Pembandingan:** Sistem menyediakan `get_model_minilm()` -> `CrossEncoder` untuk memuat model *cross-encoder/ms-marco-MiniLM-L-6-v2*. Jalur ini dapat digunakan untuk evaluasi atau perbandingan model *reranker*.
- c. **Representasi Teks Dokumen:** Menggunakan `_doc_to_rerank_text(doc: dict)` -> `str` untuk membentuk teks dokumen yang mencakup `title`, `objective`, `description`, `keywords_en`, `keywords_id`, dan `audit_indicators`. Label eksplisit seperti `Keywords EN`, `Kata kunci ID`, dan `Audit indicators` digunakan untuk memperjelas konteks.
- d. **Pembentukan Pasangan *Query-Dokumen*:** Sistem membentuk pasangan (`query`, `document_text`) untuk seluruh kandidat dalam *candidate pool*. Setiap pasangan diproses menggunakan `model.predict()` untuk menghasilkan skor mentah *cross-encoder*.
- e. **Normalisasi Skor *Reranking*:** Menggunakan `_normalize(arr: np.ndarray)` -> `np.ndarray` untuk mengubah skor *cross-encoder* ke rentang 0 sampai 1 menggunakan metode *min-max*. Jika seluruh skor sama, sistem mengembalikan vektor nol agar proses penggabungan tetap stabil.
- f. **Normalisasi Skor *Hybrid*:** Skor *hybrid* dari setiap kandidat diambil dari atribut `hybrid_score`, kemudian dinormalisasi menggunakan metode *min-max* dan disimpan sebagai `hybrid_score_norm`. Normalisasi ini dilakukan terpisah karena skala skor *hybrid* dan skor *reranking* berbeda.
- g. **Perhitungan *Combined Score*:** Sistem menghitung skor akhir menggunakan formula $combined_score = \alpha \times ce_norm + (1 - \alpha) \times hybrid_norm$. Nilai ALPHA bawaan adalah 0.8, sehingga skor *reranking* memiliki kontribusi lebih besar dibanding skor *hybrid*.
- h. **Penambahan Atribut Hasil *Reranking*:** Setiap kandidat diperkaya dengan atribut `rerank_score`, `rerank_score_norm`, `hybrid_score_norm`, dan `combined_score` untuk mendukung transparansi hasil.
- i. **Pengurutan Akhir Dokumen:** Kandidat diurutkan secara *descending* berdasarkan `combined_score`. Dokumen dengan skor tertinggi ditempatkan pada peringkat teratas.
- j. **Keluaran *Reranker*:** Hasil akhir berupa daftar kontrol ISO terbaik, yaitu top@3, yang mempertahankan informasi dari tahap *retrieval* dan menambahkan informasi

skor *reranking*. Dokumen ini kemudian digunakan sebagai konteks untuk tahap generasi jawaban.

E.3 Checklist Pemetaan Inti 4.1.4 ke 3.2.4.2

Sumber	Poin inti	Masuk ke versi usulan
4.1.4.1 Inisialisasi <i>Pipeline Reranking</i>	Inisialisasi <i>RAGPipelineV3</i> ; penggunaan korpus yang diperkaya; konfigurasi <i>candidate pool</i> dan top-k.	Ya, bagian Inisialisasi <i>Pipeline Reranking</i> .
4.1.4.2 Pembentukan <i>Candidate Pool</i>	Pemanggilan <i>hybrid retrieval</i> ; pengaturan bobot <i>dense/sparse</i> ; penyimpanan metadata dan skor kandidat.	Ya, bagian Pembentukan <i>Candidate Pool</i> .
4.1.4.3 <i>Cross-Encoder Reranking</i>	Inisialisasi model utama; model alternatif; representasi teks dokumen; pasangan <i>query</i> -dokumen; normalisasi skor <i>reranking</i> .	Ya, bagian <i>Cross-Encoder Reranking</i> .
4.1.4.4 Penggabungan Skor dan <i>Final Ranking</i>	Normalisasi skor <i>hybrid</i> ; perhitungan <i>combined score</i> ; penambahan atribut hasil; pengurutan akhir; keluaran top@3.	Ya, bagian Penggabungan Skor dan <i>Final Ranking</i> .

E.4 Rekomendasi untuk 3.2.4.2

Jika 3.2.4.2 ingin diselaraskan dengan implementasi 4.1.4, struktur dua poin saat ini sebaiknya diperluas menjadi empat bagian: inisialisasi *pipeline reranking*, pembentukan *candidate pool*, *cross-encoder reranking*, serta penggabungan skor dan *final ranking*. Namun, tingkat detailnya tetap sebaiknya lebih ringkas dibanding Bab 4. Format yang paling konsisten adalah narasi singkat pada setiap bagian, kemudian diikuti subpoin teknis ringkas agar seluruh inti 4.1.4 tetap terwakili tanpa membuat Bab 3 terlalu implementatif.